

0. STRATEGIE EXAMEN		
Sujet probable : QCM + Data Mart (Client/Produit/Vente)/Temps/Magasin) + chargement dimensions/faits + requêtes décisionnelles.		
Ordre de priorité :		
1. Reconnaître dimensions / faits / mesures		
2. Ecrire INSERT INTO ... SELECT		
3. Ecrire SELECT / JOIN / GROUP BY / HAVING		
4. Retenir pièges QCM		
1. BUSINESS INTELLIGENCE		
BI = ensemble de processus/outils pour transformer les données en informations utiles à la décision.		
Sources : ERP, CRM, fichiers, ventes, clients.		
Traitement : extraire, nettoyer, consolider, historiser.		
Consommation : reporting, dashboard, KPI, analyse.		
Besoins : centraliser, fiabiliser, consolider, KPI à jour, meilleure décision.		
KPI : CA, quantité vendue, marge, taux de retard.		
BI ne sert PAS à accélérer les transactions ni automatiser tous les métiers.		
2. OLTP vs OLAP		
Critères	OLTP	OLAP
But	Activité quotidienne.	Analysier/piloter
Données	Actuelles	Historisées
Util.	Employés	Managers/analystes
Req.	Ins/Upd/Del	SELECT/SUM/GROUP
Exemple	Paiement	CA par an
OLTP = transactions temps réel. OLAP = analyse, agrégations, historique. OLTP ne stocke PAS l'historique pour l'analyse.		
3. POURQUOI UN DATA WAREHOUSE ?		
DW = base décisionnelle orientée analyse, séparée de la base opérationnelle.		
Pourquoi pas analyser sur OLTP ? Grosses requêtes ralentissent les transactions ; données pas toujours nettoyées/historisées ; sources dispersées, formats différents.		
4 propriétés Inmon :		
● Subject-oriented : organise par sujets métier (client, produit, vente), pas par application.		
● Integrated : sources multiples unifiées (noms, formats, codes, devises).		
● Time-variant : garde l'historique ; analyse l'évolution dans le temps.		
● Nonvolatile : on charge et consulte ; on ne modifie pas sans passer comme OLTP.		
Single version of truth : un seul chiffre officiel partagé.		
DW n'est PAS fait pour saisir les ventes quotidiennes ; il sert à les analyser.		
4. DATA MART		
Data Mart = sous-ensemble spécialisée d'un DW, dédiée à		

un domaine (ventes, RH, finance).
Ex. Data Mart commercial : Dim_Produit, Dim_Client, Dim_Temps, Fait_Vente.
DW = global et reprise **Data Mart** = service/domaine.

5. ETL / ELT / STAGING
E (Extract) : lire depuis ERP, CRM, Excel, BD, fichiers.
T (Transform) : nettoyer, convertir, uniformiser, calculer.
L (Load) : charger dans staging puis DW/Data Mart.
Transformations typiques : doublons, NULL, dates décomposées (jour/mois/année), devises, sexe M/F, montant = quantité*Prix, mapping des jointures avec dimensions.
Staging area : zone temporaire brute/intermédiaire avant DW.
Talend/SSIS : outils ETL visuels (source, filtre, agrégation, mapping, jointure, destination).
ETL ne fait PAS les transactions métiers. ETL ne calcule PAS automatiquement les dimensions.

6. MODELISATION DIMENSIONNELLE		
Table de faits		
Ce qu'on mesure. Contient mesures + clés et rangées (FK) vers dimensions. Ex. : <code>Fait_Vente(id_vente, id_produit, id_client, id_date, id_magasin, quantite/ventes, montant/ventes)</code> . Beaucoup de lignes, peu de colonnes.		
Table de dimensions		
Axe d'analyse descriptif. Ex. : Produit(nom, catégorie), Client(ville, segment), Temps(jour, mois, année). Peu de lignes, beaucoup de colonnes.		
Contient : Surrogate Key (SK) = clé technique artificielle, indépendante de la clé métier, utile pour historiser.		
Natural Key (NK) = clé originale du système source.		
Mesures		
Nombre à calculer/agréger : quantité, montant, CA, cout, marge.		
● Additives : SUM sur toutes dimensions (quantité, montant, CA).		
● Semi-additives : SUM valide sur certaines dim. seulement (stock, solde bancaire – pas sur le temps).		
● Non-additives : ratio/pourcentage/taux. Recalculer depuis numérateur/dénominateur. JAMAIS SUM direct.		
Grain		
Niveau exact d'une ligne de la table de faits. Ex. : une ligne = une vente d'un produit à un client dans un magasin à une date. Doit être défini avant construction. Détermine la précision max des analyses.		
Méthode de reconnaissance		
Champ numérique à analyser = mesure/faît. Champ descriptif pour filtrer/grouper = dimension. Date = Dim_Temps.		

id_produit dans Fait_Vente n'est PAS une mesure, c'est une FK.

7. ETOILE, FLOCON, CONSTELLATION

Etoile (Star) : table de faits au centre, dimensions autour dénormalisées. Simple, peu de jointures, rapide. **Tres probable à l'examen.**

Flocon (Snowflake) : dimensions normalisées en sous-dimensions (Produit → Catégorie). Moins redondant, mais plus de jointures, plus complexe. Non recommandé juste pour économiser l'espace.

Constellation (Galaxy) : plusieurs tables de faits partageant des dimensions. Ex : Fait_Vente + Fait_Livraison.

	Etoile	Flocon	Gal.
Faits	1	1	Plusieurs
Dim.	Plates	Norm.	Partag.
Jointures	Peu	Bcp	Variable
Vitesse	Rapide	Lent	Variable



8. AJOUTER DIM vs NOUVELLE TABLE FAITS		
Ajouter une dimension si : nouvel axe d'analyse relie au même processus et au même grain. Conditions : utile, disponible, qualité ok, relation logique, attributs riches. Ex. : chaque vente a un magasin → ajouter Dim_Magasin et FK id_magasin dans Fait_Vente.		
Nouvelle table de faits si : nouveau processus métier OU grain différent OU dimensions totalement différentes. Ex. : ventes (grain vente) vs livraisons (grain colis) vs stocks (snapshot journalier).		
9. ROLAP / MOLAP / HOLAP		
ROLAP : données dans SGBD relationnel (étoile/flocon), requêtes SQL générées. Grande taille, moins rapide sur gros agrégats.		
MOLAP : cubes multidimensionnels, pré-calculs. Très rapide pour analyse interactive, limites de taille.		
HOLAP : détail dans relationnel + agrégats dans cubes. Combine ROLAP et MOLAP.		
Tech.	Vitesse	Taille
ROLAP	Lent	Illimitée
MOLAP	Tres rapide	Limitée
HOLAP	Satisfaisant	Grande
MOLAP plus rapide que ROLAP pour les agrégats. ERR D QCM.		
Operations OLAP : Slice = tranche (1 dim). Dice = sous-cube (N dim). Roll-up = resumer (jour→mois→année). Drill-down = détailler (année→mois→jour).		

10. REGLES D'ASSOCIATION	
But : trouver liens/correlations entre produits, comportements d'achat, cross-selling, recommandations.	
Ex. : clients qui achètent PC achètent souvent souris.	
● Support : fréquence de A et B ensemble dans toutes les transactions.	
● Confiance : parmi ceux qui ont A, proportion qui ont aussi B = P(B A).	
● Cross-selling : recommander B si A acheté.	
Technique	Usage
Regles asso.	Corrélations, paniers
Regression	Predire valeur continue
Clustering	Segmenter des groupes
Classification	Predire une catégorie
Support + Confiance UNIQUEMENT pour règles d'association. Accuracy, CTR, Gain = autres domaines.	

11. QCM ANNALE — REPONSES RAPIDES	
Q1. Besoins BI	
OK A : Centraliser/fiabiliser données multi-sources	
OK C : Faciliter décisions stratégiques	
OK E : Obtenir KPI pertinents et à jour	
Q2. OLTP / OLAP	
OK A : OLTP gère opérations temps réel	
OK B : OLAP analyse multidimensionnelle	
OK D : OLAP optimise calculs/agrégations	
ERR C : C'est OLAP (pas OLTP) qui stocke l'historique	
Q3. ETL	
OK A : Extraît sources (ERP, CRM, Excel)	
OK C : Transforme, nettoie, normalise	
OK D : Charge les données dans DWH	
ERR B : ETL n'exécute PAS les transactions métiers	
ERR E : ETL ne calcule PAS les dimensions	

Q4. Etapes système décisionnel	
OK A : Identifier besoins métiers + KPI	
OK C : Concevoir modèle dimensionnel	
OK D : Mettre en place ETL	
OK E : Déployer outils visualisation/reporting	
ERR B : On ne construit PAS une BD relationnelle transactionnelle pour le décisionnel	

Q5. Critères ajouter une dimension	
OK A : Utile pour analyses métier	
OK B : Données disponibles et de qualité	
OK C : Relation logique avec table de faits	
OK D : Attributs riches et hiérarchiques	
Q6. Nouvelle table de faits	
OK A : Processus métier différent	
OK B : Dimensions totalement différentes	
OK C : Indicateur pas exprimable au même grain	

Q7. ROLAP / MOLAP / HOLAP	
OK A : ROLAP = base relationnelle	
OK B : MOLAP = cubes multi-idim.	
OK C : HOLAP = hybride	
OK E : MOLAP optimal pour analyses rapides	
ERR D : MOLAP PLUS rapide que ROLAP (pas l'inverse)	
Q8. Regles d'association	
OK A : Corrélations entre produits/comportements	
OK C : Paniers d'achat fréquents	
OK D : Liens cachés entre événements	
ERR B : Predire variable continue = regression	

Q9. Métriques règles d'association	
OK A : Support	OK B : Confiance
ERR C/D/E : Precision, Gain, CTR = autres domaines	
Q10. Applications règles d'association	
OK A : Recommandation produits (cross-selling)	
OK C : Analyse comportements d'achat	
ERR B : Prévision ventes = regression. ERR D : Segmentation = clustering	
Q11. Talent	
OK B : Données à jour, consolidées, disponibles	
OK D : Transformer toutes variétés de données	
ERR A : pas TOUS les besoins. ERR C : pas les transactions	
Q12. Composantes Talent	
OK A : Modification	OK B : Filtrage
OK C : Agrégation	OK D : Intégration

12. DICTIONNAIRE EXPRESS	
KPI : indicateur de performance.	
Dashboard : tableaux de bord.	
Reporting : rapports.	
Granularité fine : détail. Grossière : résumé.	
Ad hoc : requête non prévue à l'avance.	
Historisation : garder les anciennes valeurs.	
Single version of truth : chiffre officiel partagé.	
Data-driven decision : décision basée sur données.	
SCD : Slowly Changing Dimension.	
● Type 1 : efface l'ancienne valeur.	
● Type 2 : ajoute une nouvelle ligne (historique complet).	
● Type 3 : ajoute un attribut ancien/nouveau.	

13. CREATE TABLE (si demande)	
<pre>CREATE TABLE Dim_Client (id_client INT PRIMARY KEY, nom VARCHAR(50), prenom VARCHAR(50), ville VARCHAR(50), age INT, genre VARCHAR(20)); CREATE TABLE Dim_Produit (id_produit INT PRIMARY KEY, nom_produit VARCHAR(100), categorie VARCHAR(50), prix DECIMAL(10,2)); CREATE TABLE Dim_Temps (id_date INT PRIMARY KEY, date DATE, jour INT, mois INT, annee INT); CREATE TABLE Dim_Magasin (id_magasin INT PRIMARY KEY, nom_magasin VARCHAR(100)); CREATE TABLE Fact_Vente (id_vente INT PRIMARY KEY, id_produit INT, id_client INT, id_magasin INT, id_date INT, quantiteVentes INT, montantVentes DECIMAL(10,2), FOREIGN KEY(id_produit) REFERENCES Dim_Produit(id_produit), FOREIGN KEY(id_client) REFERENCES Dim_Client(id_client), FOREIGN KEY(id_magasin) REFERENCES Dim_Magasin(id_magasin), FOREIGN KEY(id_date) REFERENCES Dim_Temps(id_date));</pre>	

14. T-SQL — STRUCTURE & RECAP	
Ordre execution : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY	

<pre>SUM(s) AVG(s) COUNT(*) COUNT(DISTINCT id) MIN(s) / MAX(s)</pre>	Somme Moyenne N b lignes N b distincts Min / Max
Fonctions d'agregation	
WHERE : filtre les lignes avant agrégation. HAVING : filtre les groupes après agrégation.	
WHERE et AVG : 2021 GROUP BY id, nom, produit, an, nom_magasin HAVING SUM(f.montantVentes) > 10000	
JAM AIS WHERE SUM(...) > 10000. Correct : HAVING SUM(...) > 10000	
Sous-requêtes	
JAM AIS WHERE Price = MAX(Price) – utiliser sous-requête ! WHERE sSalesItem = (SELECT MAX(SalesItem) FROM Sales.SalesItem); WHERE sSalesItem < (SELECT AVG(SalesItem) FROM Sales.SalesItem);	
Fonctions dates	
YEAR(d) / MONTH(d) / DAY(d)	Entiers
DATETIMEPART (YEAR, d)	Entier
DATEDIFF (s, d1, d2)	Différence
DATEDIFF (s, n, d)	Ajoute n unités
GETDATE()	Date/heure serveur
Fonctions chaînes	
UPPER(s) / LOWER(s)	Casse
LEFT(s, n) / RIGHT(s, n)	n car.
SUBSTRING (s, d, l)	Extrait
CONCAT (val, d2)	Concatène
REPLACE (s, old, new)	Remplace
LEN(s)	Longueur
Fonctions math + CASE + COALESCE	
ROUND (n, dec)	Arrondi
CEILING (n) / FLOOR (n)	Sup / Inf
CAST (e AS type)	Conversion
<pre>-- CASE / IF / IIF / IS SELECT CASE CreditRating WHEN 1 THEN 'superieur' WHEN 2 THEN 'moyenn' ELSE 'autre' END AS Libelle FROM Purchasing.Vendor; -- COALESCE : 1st non-NULL SELECT COALESCE(psc.Name, 'CREATE (Pas de nom) - cat : ' + p.Name) AS SC FROM Production.Product p LEFT JOIN Production.ProductSubcategory psc ON p.ProductSubcategoryID = psc.ProductSubcategoryID;</pre>	
DISTINCT / TOP / UNION	
SELECT DISTINCT (Status) FROM Sales.SalesOrder; SELECT TOP 10 * FROM Production.Product; SELECT TOP 1 ProductID, Name, SUM(OrderQty) FROM ... GROUP BY ProductID, Name ORDER BY 2 DESC; SELECT City FROM Customer; INTERSECT SELECT City FROM Supplier;	
JOIN = fusion horizontale (colonnes). UNION = fusion verticale (lignes, même nb colonnes).	
VUES	
<pre>CREATE VIEW vw_Ren AS SELECT col1, col2 FROM t1 JOIN t2 ON ... -- Utiliser come une table : SELECT * FROM vw_Ren WHERE col1 = 'val'; DROP VIEW vw_Ren; -- Supprime sources concernées</pre>	
Pièces T-SQL recapitulatif	
Erreur	Correct
WHERE P=MAX(P)	Sous-requête
WHERE COUNT(.) > 10	HAVING
SELECT Sum, SUM(s)	*GROUP BY Sum
JOIN pour empiler lignes	UNION

15. METHODE REQUETE DECISIONNELLE 9 étapes dans l'ordre : 1. Lire la question tout entierement 2. Repérer les mesures demandées (SUM, AVG, COUNT) 3. Repérer les axes d'analyse (par produit, par année, par magasin,...) 4. Choisir la table de faits (point de départ) 5. Joindre les dimensions nécessaires (JOIN) 6. Placer les filtres avant aggregation (WHERE) 7. Grouper par les axes (GROUP BY) 8. Filtrer les résultats agreg. (HAVING) 9. Trier si demandé de (ORDER BY) Correspondances : • Quantité totale → SUM(f.quantitéVentes) • Montant total → SUM(f.montantVentes) • Moyenne → AVG(...) • Nombre → COUNT(...) • Par produit/magasin/année → SELECT + GROUP BY • année = 2025 → WHERE dt.année = 2025 • Montant total > 10000 → HAVING SUM(...) > 10000 16. TEMPLATE SELECT DECISIONNEL <pre>SELECT -- ex: dp.non_produit amei, -- ex: dn.non_magasin amei, -- ex: dn.non_magasin SUM(f.meure) AS total_meure, SUM(f.meure2) AS total_meure2 FROM Fait_Vente f JOIN Dim_Produit dp ON f.f_id_produit = dp.id_produit JOIN Dim_Client dc ON f.f_id_client = dc.id_client JOIN Dim_Temps dt ON f.f_id_date = dt.id_date JOIN Dim_Magasin dm ON f.f_id_magasin = dm.id_magasin WHERE condition_avant_agregation GROUP BY amei, ame2 HAVING condition_apres_agregat ORDER BY amei;</pre> Toute colonne non agrégée dans SELECT DOIT être dans GROUP BY. WHERE ne contient JAMAIS SUM/AVG/HAVING content SUM/AVG. 17. SCHEMA ETOILE TYPE EXAMEN Modele operationnel (source) Client(id_client, nom, prenom, ville, segment) Produit(id_produit, nom, categorie, prix) Vente(f_id_vente, id_client, id_client, date_vente, quantite, prix_unitaire) Magasin(id_magasin, nom_magasin) Modele decisionnel etoile Dim_Produit (id_produit PK, nom_produit, categorie, prix) Dim_Client (id_client PK, nom, prenom, ville, segment) Dim_Temps (id_date PK, date_jour, jour, mois, année) Dim_Magasin (id_magasin PK, nom_magasin) Fait_Vente (f_id_vente PK, id_client FK, id_date FK, id_magasin FK, quantiteVentes, montantVentes) Mesures : quantiteVentes, montantVentes = quantite * prix_unitaire. Axes : produit/categorie, client/segment, temps/année, magasin. 18. CHARGEMENT DIMENSIONS (INSERT) <pre>-- Dim_Produit INSERT INTO Dim_Produit (id_produit, nom_produit, categorie, prix) SELECT id_produit, nom, categorie, prix FROM Produit;</pre> 20. REQUETES DECISIONNELLES COMPLETES	Par produit, magasin et année <pre>SELECT dp.nom_produit, dn.nom_magasin, dt.année, SUM(f.quantiteVentes) AS quantite_totale, SUM(f.montantVentes) AS montant_total FROM Fait_Vente f JOIN Dim_Produit dp ON f.f_id_produit=dp.id_produit JOIN Dim_Magasin dm ON f.f_id_magasin=dm.id_magasin JOIN Dim_Temps dt ON f.f_id_date =dt.id_date GROUP BY dp.nom_produit, dn.nom_magasin, dt.année;</pre> Année 2025 par produit et magasin <pre>SELECT dp.nom_produit, dn.nom_magasin, SUM(f.quantiteVentes) AS quantite_totale, SUM(f.montantVentes) AS montant_total FROM Fait_Vente f JOIN Dim_Produit dp ON f.f_id_produit=dp.id_produit JOIN Dim_Magasin dm ON f.f_id_magasin=dm.id_magasin JOIN Dim_Temps dt ON f.f_id_date =dt.id_date WHERE dt.année = 2025 GROUP BY dp.nom_produit, dn.nom_magasin HAVING SUM(f.montantVentes) > 10000;</pre> année=2025 est un filtre avant agregation : WHERE, pas HAVING. Montant total > 10 000 en 2025 <pre>SELECT dp.nom_produit, dn.nom_magasin, SUM(f.quantiteVentes) AS quantite_totale, SUM(f.montantVentes) AS montant_total FROM Fait_Vente f JOIN Dim_Produit dp ON f.f_id_produit=dp.id_produit JOIN Dim_Magasin dm ON f.f_id_magasin=dm.id_magasin JOIN Dim_Temps dt ON f.f_id_date =dt.id_date WHERE dt.année = 2025 GROUP BY dp.nom_produit, dn.nom_magasin HAVING SUM(f.montantVentes) > 10000;</pre> Montant > moyenne generale <pre>SELECT dp.nom_produit, dn.nom_magasin, SUM(f.montantVentes) AS montant_total, SUM(f.montantVentes) AS montant_total FROM Fait_Vente f JOIN Dim_Produit dp ON f.f_id_produit=dp.id_produit JOIN Dim_Magasin dm ON f.f_id_magasin=dm.id_magasin JOIN Dim_Temps dt ON f.f_id_date =dt.id_date WHERE dt.année = 2025 GROUP BY dp.nom_produit, dn.nom_magasin HAVING SUM(f.montantVentes) > (SELECT AVG(f.montantVentes) FROM Fait_Vente f2 JOIN Dim_Temps dt2 ON f2.f_id_date=dt2.id_date WHERE dt2.année = 2025);</pre> Par catégorie > 50 000 en 2025 <pre>SELECT dp.categorie, dt.année, SUM(f.montantVentes) AS chiffre_affaires FROM Fait_Vente f JOIN Dim_Produit dp ON f.f_id_produit=dp.id_produit JOIN Dim_Temps dt ON f.f_id_date =dt.id_date WHERE dt.année = 2025 GROUP BY dp.categorie, dt.année HAVING SUM(f.montantVentes) > 50000 ORDER BY chiffre_affaires DESC;</pre> Raisonnement : mesure = montant ; axes = categorie + année ; filtre avant = 2025 (WHERE) ; filtre apres = CA > 50000 (HAVING).
--	--

21. REQUETES T-SQL AVANCEES (AdventureWorks) Agreg. par année avec DatePart <pre>SELECT DATEDIFF(YEAR, OrderDate) AS année, SUM(TotalDue) AS total FROM Sales.SalesOrderHeader GROUP BY DATEDIFF(YEAR, OrderDate) ORDER BY année;</pre> TOP / PERCENT meilleur produit <pre>SELECT TOP 1 PERCENT p.Name, SUM(od.OrderQty) AS Total FROM Sales.SalesOrderDetail od JOIN Production.Product p ON od.ProductID = p.ProductID GROUP BY p.Name ORDER BY 2 DESC;</pre> Sous-catégories avec plus de 10 produits <pre>SELECT ps.Name, COUNT(p.ProductID) AS Nb FROM Production.Product ps JOIN Production.ProductSubcategory psc ON ps.ProductSubcategoryID = psc.ProductSubcategoryID GROUP BY ps.Name HAVING COUNT(p.ProductID) > 10;</pre> Code utilisateur : 3 lettres nom + 1 prenom <pre>SELECT DISTINCT LastName, FirstName, UPPER(CONCAT(SUBSTRING(LastName,1,3), SUBSTRING(FirstName,1,3))) AS Code FROM Person.Person;</pre> DATEDIFF - nb jours depuis commande <pre>SELECT OrderDate, DATEDIFF(OFF, OrderDate, GETDATE()) AS NbJours FROM Sales.SalesOrderHeader;</pre> CASE - traduire code en libellé <pre>SELECT CreditRating, CASE CreditRating WHEN 1 THEN 'Superieur' WHEN 2 THEN 'Excellent' WHEN 3 THEN 'Au-dessus moy.' WHEN 4 THEN 'Moyenne' ELSE 'Sous la moyenne' END AS Libelle, COUNT(*) AS NbClients FROM Purchasing.Vendor GROUP BY CreditRating;</pre> COALESCE - gerer les NULL <pre>SELECT p.Name AS Produit, COALESCE(pac.Name, CONCAT('Pas de SC', p.Name)) AS SC FROM Production.Product p LEFT JOIN Production.ProductSubcategory pac ON p.ProductSubcategoryID = pac.ProductSubcategoryID;</pre> INTERSECT / UNION / EXCEPT <pre>-- Filtré dans les deux tables SELECT City FROM Customer INTERSECT SELECT City FROM Supplier; -- Filtré dans Customer seulement SELECT City FROM Customer EXCEPT SELECT City FROM Supplier; -- Calculer total + nombre SELECT 'Total : ';</pre>	+ CAST(COUNT(*) AS NVARCHAR(10)) + ' commandes en 2014.' FROM Sales.SalesOrderHeader WHERE YEAR(OrderDate) = 2014; DISTINCT sur COUNT <pre>SELECT COUNT(DISTINCT CustomerID) AS NbClients FROM Sales.SalesOrderHeader;</pre> 22. ARCHITECTURE BI COMPLETE <table><tr><th>Couche</th><th>Contenu</th></tr><tr><td>1 – Sources</td><td>OLTP, ERP, CRM, fichiers</td></tr><tr><td>2 – DWH</td><td>ETL (Extract, Transform, Load) Modèle dimensionnel Data Marts (sous-ens. métier)</td></tr><tr><td>3 – Analyse</td><td>OLAP, Reporting, Mining Décideurs et analystes</td></tr></table> Cas Continental Airlines : 5e compagnie US, 50M passagers. Problème : silos de données, faille. Solution : Go-Forward Plan (30M\$, 6 ans). Resultat : N°1 en 2004, ROI >1000%. 23. SYNTHESE SCHEMAS & CONCEPTS CLES Surrogate Key vs Natural Key SK : PK artificielle genere par DWH (pas du systeme source). Permet de gerer l'historique. NK : ID original du systeme source. Peut changer ou etre remplie. Dimension degenerate Dimension stockée directement dans la table de faits (pas de table separée). Ex : N° de commande, N° de ticket. Utile quand la dimension n'a pas d'attributs descriptifs. Mesures – Rappel critique Additive : SUM sur toutes dimensions (quantité, revenu, CA). Semi-additive : SUM sur certaines dim. seulement (stock, stock). Non-additive : JAMAIS SUM (taux, ratio, prix unitaire). Recalculer depuis numerateur/denominator. Regles d'association – Formules Support(A,B) = transactions(A∩B) / nb total transactions. Confiance(A→B) = transactions(A∪B) / transactions(A). 24. CHECKLIST AVANT DE RENDRE QCM : • Lire toutes les propositions – plusieurs vraies possibles. • OLAP stocke l'historique (pas OLTP). • MOLAP plus rapide que ROLAP. • ETL ne fait pas les transactions métiers. • Regles asso : metriques = Support + Confiance seulement. • Clustering = segmentation. Regression = prediction valeur. Schema dimensionnel : • Faits au centre, dimensions autour. • Mesures dans la table de faits. • Attributs descriptifs dans les dimensions. • FK dans Fait, PK dans Dim. Requetes decisionnelles : • Chaque axe dans SELECT doit etre dans GROUP BY. • WHERE avant GROUP BY ; HAVING apres. • Pour Dim_Temps : DISTINCT + DAY/MON- TH/YEAR. • Pour Fait : partir de Ventes v, puis JOIN dimensions. • montantVentes = quantite * prix_unitaire. • Si ajout magasin : Dim_Magasin+ id_magasin FK + JOIN. • Ecrire aliases : f, dp, dc, dt, dm. Si bloque : ecrire le squelette SELECT / FROM / JOIN / WHERE / GROUP BY puis remplir. 25. ANTI-PIEGES QCM RECAPITULATIF <table><tr><th>Affirmation FAUSSE</th><th>Ce qui est VRAI</th></tr><tr><td>OLTP stocke l'historique</td><td>C'est OLAP</td></tr><tr><td>ETL execute transactions</td><td>Non, jamais</td></tr><tr><td>Decisionnel = BD 3NF</td><td>Modele dimensionnel</td></tr><tr><td>ROLAP plus rapide MO-</td><td>C'est l'inverse</td></tr><tr><td>WHERE COUNT(.)>10</td><td>HAVING COUNT(.)>10</td></tr><tr><td>WHERE p = NAI(p)</td><td>Sous-requete</td></tr><tr><td>SUM(taux) possible</td><td>Jamais SUM(taux)</td></tr><tr><td>JOIN pour emplier lignes</td><td>UNION</td></tr></table> 26. FORMULES A CASER EN REPONSE Table de faits : contient les mesures et les cles etrangeres « vers les dimensions » elle represente un evenement metier mesurable. Dimension : point de vue d'analyse avec attributs descriptifs. Grain : niveau exact d'une ligne de la table de faits. Surrogate key : cle technique pour remplacer une cle metier et mieux historiser. Single version of truth : chiffres fiables et partagees par toute l'ent reprise. Data-driven decision : decision basee sur donnees, pas sur intuition. ETL : Extract (sources) → Transform (nettoyer, standardiser) → Load (charger dans DWH). Data Mart : sous-ensemble du DWH oriente vers un metier specifique. 27. RECAP ULTRA-FINAL — REGLES D'OR T-SQL • Agregat dans WHERE → sous-requete • Filtrer un groupe → HAVING • Colonne SELECT non agrégée → GROUP BY • Empiler lignes → UNION ; colonnes → JOIN • VIEW = SELECT sauvegardée, s'utilise comme une table • COALESCE(a,b) = a si non-NULL, sinon b DWH & Modelisation • Table faits : FK + mesures, beaucoup de lignes • Table dim : SK + NK + attributs, beaucoup de colonnes • Etiole = rapide (denorm.) ; Flocon = lent (norm.) • DWH : LOAD + SELECT seulement (jamais UP-DATE/DELETE) • Mesure additive : SUM partout ; non-additive : recalculer • Granularité fine = plus de lignes = plus de détail QCM • OLAP ≠ OLTP : l'historique appartient a OLAP • MOLAP > ROLAP en vases ; ROLAP > MOLAP en taille • Regles asso : Support + Confiance (rien d'autre) • Regression ≠ association ; Clustering ≠ association • ETL : Extract + Transform + Load (pas de transactions) • Talend : OK pour transformation/chargement ; NON pour transactions	Couche	Contenu	1 – Sources	OLTP, ERP, CRM, fichiers	2 – DWH	ETL (Extract, Transform, Load) Modèle dimensionnel Data Marts (sous-ens. métier)	3 – Analyse	OLAP, Reporting, Mining Décideurs et analystes	Affirmation FAUSSE	Ce qui est VRAI	OLTP stocke l'historique	C'est OLAP	ETL execute transactions	Non, jamais	Decisionnel = BD 3NF	Modele dimensionnel	ROLAP plus rapide MO-	C'est l'inverse	WHERE COUNT(.)>10	HAVING COUNT(.)>10	WHERE p = NAI(p)	Sous-requete	SUM(taux) possible	Jamais SUM(taux)	JOIN pour emplier lignes	UNION
Couche	Contenu																										
1 – Sources	OLTP, ERP, CRM, fichiers																										
2 – DWH	ETL (Extract, Transform, Load) Modèle dimensionnel Data Marts (sous-ens. métier)																										
3 – Analyse	OLAP, Reporting, Mining Décideurs et analystes																										
Affirmation FAUSSE	Ce qui est VRAI																										
OLTP stocke l'historique	C'est OLAP																										
ETL execute transactions	Non, jamais																										
Decisionnel = BD 3NF	Modele dimensionnel																										
ROLAP plus rapide MO-	C'est l'inverse																										
WHERE COUNT(.)>10	HAVING COUNT(.)>10																										
WHERE p = NAI(p)	Sous-requete																										
SUM(taux) possible	Jamais SUM(taux)																										
JOIN pour emplier lignes	UNION																										